Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación
IIC3745: Testing

Tarea 2: Agentic Program Repair

Juan Pablo Sandoval, José Antonio Tapia
Septiembre-2026

1. Objetivos

Al finalizar esta tarea, deberán ser capaces de:

- Diseñar un sistema basado en LLM organizado en etapas (*pipeline*), con entradas y salidas bien definidas entre ellas.
- Seleccionar qué contexto entregarle al LLM (un repositorio completo no cabe en un prompt) y medir su costo en tokens.
- Utilizar señales deterministas (ejecución de tests, trazas de error, estructura del código) para guiar al LLM.
- Verificar automáticamente la salida del LLM (tests objetivo y regresiones) y reintentar con retroalimentación.
- Medir un sistema: tasa de éxito, calidad de la localización, costo y tiempo.
- Construir una solución que **generalice** a bugs no vistos durante el desarrollo.

2. Parte I – Implementación del Agente de Reparación

En la Tarea 1 utilizamos un LLM para *generar* pruebas. En esta tarea invertimos el problema: partimos de un proyecto real que tiene un bug y de un conjunto de tests que lo exponen (fallan), y el objetivo es que un agente **modifique el código fuente** para que esos tests pasen, sin romper el resto del proyecto. Esta área se conoce como *Automated Program Repair* (APR).

Reparar un bug con un LLM presenta desafíos propios. Un repositorio real tiene miles de líneas y no cabe completo en la ventana de contexto del modelo (y, aunque cupiera, sería caro). Por lo tanto, el agente primero debe descubrir *dónde* está el problema y luego decidir *qué* fragmento de código mostrarle al modelo. Además, el modelo puede proponer cambios que no se pueden aplicar, que rompen la sintaxis, que hacen pasar el test objetivo pero rompen otros tests (*regresiones*), o que simplemente no corrigen nada. El agente debe detectar estos casos y reaccionar.

Por esto, el agente se organiza obligatoriamente en **dos etapas separadas**:

1. **Localizador:** a partir del repositorio y los tests que fallan, determina las regiones del código fuente donde probablemente está el bug.
2. **Reparador:** recibe como entrada la localización, genera una modificación del código, la aplica y verifica que los tests pasen sin regresiones.

Los bugs corresponden a bugs reales de proyectos Python de código abierto, preparados para esta tarea con scripts uniformes para instalar el entorno y ejecutar los tests.

Generalización. Su agente será evaluado también con **bugs que ustedes no han visto** de los mismos proyectos, empaquetados con el mismo formato. Un agente ajustado a mano a los bugs públicos (nombres de archivos, funciones o tests concretos, heurísticas pensadas para un bug o proyecto) rendirá mal en esa evaluación y además será penalizado.

¿Qué entendemos por regresión? Una **regresión** ocurre cuando un cambio en el código hace que deje de funcionar algo que antes funcionaba. En esta tarea, significa que el agente corrige el bug (los tests que lo exponen pasan), pero su modificación hace fallar **otros tests del proyecto que antes pasaban**. Por ejemplo, si el agente corrige cómo se formatea una instrucción `return` y, de paso, rompe el formateo de las expresiones `lambda`, un test de `lambda` que antes pasaba ahora fallará: el bug se "arregló", pero se introdujo un problema nuevo.

Por eso no basta con que pasen los tests del bug. En concreto, para cada bug:

- Los **tests objetivo** son los que exponen el bug (`comandos_test` en `bug.json`): fallan con el código original y deben pasar después de la reparación.
- Los **tests de regresión** son el resto de los tests de los módulos listados en `modulos_test`: todos los que pasan con el arreglo correcto deben **seguir pasando** después de la reparación del agente.

Una reparación que hace pasar los tests objetivo pero rompe algún test de regresión se considera **no aprobada**. Su agente debería detectar esta situación (ejecutando los módulos de test tras aplicar el cambio) y reaccionar, por ejemplo, revirtiendo el cambio y reintentando.

2.1. Requisitos del agente

El agente desarrollado deberá ser capaz de:

- Leer los metadatos del bug (`bug.json`) y ejecutar los tests que lo exponen.
- Analizar la salida de los tests que fallan (*traceback*, mensajes de aserción, etc.) y la estructura del repositorio.
- **Localizar** el bug y reportarlo en `localizacion.json`.
- Seleccionar y acotar el contexto que se envía al LLM, controlando el tamaño de los prompts.
- **Generar y aplicar** una reparación sobre el código fuente del repositorio.
- **Verificar** la reparación: los tests objetivo deben pasar y no deben aparecer regresiones en los módulos de test relacionados.
- Reintentar con retroalimentación (errores, tests que siguen fallando, regresiones) cuando la reparación no funcione.
- Manejar errores temporales de la API (por ejemplo, 429 y 503) con reintentos y espera creciente, y respuestas inválidas del modelo, sin caerse.
- Contabilizar llamadas, tokens y tiempo, y reportarlos en `metricas.json`.

- **Time Budget:** la ejecución del agente sobre **cada bug** (`python main.py <bug>`) no puede superar los **3 minutos por bug**, sin contar la instalación del entorno (`setup.sh`), que se realiza antes y por separado. El agente debe incorporar un control de tiempo y terminar dejando sus resultados escritos antes de ese límite. Si no termina a tiempo, el bug cuenta como no reparado.

2.2. Componentes esperados del agente

No se exige ninguna técnica en particular, pero su agente debe contar con las siguientes piezas. Para cada una se evaluará que **exista, funcione y que el grupo sepa explicarla**. La tabla indica qué problema resuelve cada pieza y el mínimo que debe cumplir; **cómo** resolverlo es parte del diseño de cada grupo.

Pieza	**Problema que resuelve**	**Mínimo**
1. Orquestación (`main.py`)	Coordinar las etapas y asegurar que siempre queden escritos los resultados.	Ejecuta localizador → reparador y escribe ambos JSON con el formato exigido.
2. Ejecución e interpretación de tests	Saber qué falla y obtener información útil de esa falla.	Ejecuta los tests del bug y distingue si pasan o fallan.
3. Señales para localizar	Decidir dónde mirar sin revisar todo el repositorio.	Usa al menos una señal obtenida de los tests (su salida o su código) para acotar dónde buscar.
4. Construcción del contexto	Decidir qué información recibe el LLM y cuánta.	Envía al LLM el código de la región sospechosa y la información de la falla.
5. Formato de respuesta y aplicación del cambio	Convertir la respuesta del LLM en un cambio real sobre el código.	Pide una respuesta procesable y la aplica al código fuente (nunca a los tests).
6. Verificación	Saber si la reparación realmente funciona.	Tras aplicar el cambio, ejecuta los tests del bug.
7. Reintentos con retroalimentación	Qué hacer cuando la reparación no funciona.	Reintenta cuando la reparación falla.
8. Robustez ante la API	Seguir funcionando cuando la API falla o responde algo inválido.	No se cae ante un error de la API.
9. Medición del costo	Saber cuánto cuesta reparar cada bug.	Cuenta llamadas, tokens y tiempo en `metricas.json` según la Sección 2.3.
10. Generalización	Funcionar con bugs y proyectos no vistos.	No hay nada específico de un bug o proyecto en el código.

El mínimo asegura un agente funcional; los mejores resultados en efectividad y generalización requieren ir más allá de él.

2.3. Input y Output

El agente se ejecuta sobre un paquete de bug mediante:

```
cd agente
python main.py ../dataset/<proyecto>-<n>
```

Formato de cada paquete de bug:

```
black-12/
|-- repo/              código del proyecto con el bug (git con un único commit)
|-- bug.json           metadatos del bug
|-- requirements.txt   dependencias del proyecto en la versión del bug
|-- setup.sh           crea el entorno del bug en repo/env (Python 3.8)
`-- run_test.sh        ejecuta los tests que exponen el bug (código 0 = pasan)
```

Campos de `bug.json`:

Campo	Descripción
`id`, `proyecto`, `python`	Identificador, proyecto y versión de Python (3.8.3).
`comandos_test`	Comandos que ejecutan los tests que fallan (los que usa `run_test.sh`).
`archivos_test`	Archivos de test relacionados con el bug (incluye datos de test).
`modulos_test`	Módulos de test completos, útiles para detectar regresiones.

Al terminar, deben quedar en la carpeta del bug:

1. **La reparación aplicada en `repo/`.** Se evalúa el estado final de `repo/`.
2. **`localizacion.json`**, con las regiones sospechosas ordenadas de mayor a menor sospecha:

   ```
   {"regiones": [{"archivo": "black.py", "inicio": 120, "fin": 145}]}
   ```

   `archivo` es relativo a `repo/` y debe ser código fuente (no un test); `inicio` y `fin` son números de línea (desde 1, inclusivos) del archivo *con el bug*. Pueden agregar campos extra.
3. **`metricas.json`**, con exactamente estos campos (valores de ejemplo):

   ```
   {"bug": "black-12", "modelo": "gemini-3.5-flash-lite", "llamadas": 2,
    "tokens_entrada": 3634, "tokens_salida": 448, "segundos": 131.4}
   ```

   - `tokens_entrada` / `tokens_salida`: suma de `usage_metadata.prompt_token_count` y `candidates_token_count`, respectivamente, de todas las respuestas del modelo.
   - `llamadas`: número de llamadas que obtuvieron respuesta (sin contar reintentos fallidos).
   - `segundos`: tiempo total del agente en el bug, incluida la ejecución de tests.

Pueden agregar módulos, dependencias y campos adicionales, pero **no pueden cambiar el comando de ejecución ni estos formatos**, ya que se utilizan para la evaluación automática.

2.4. Dataset

Todos los bugs usan **Python 3.8.3** y se ejecutan en **Linux, macOS o WSL**. Se entregan **12 bugs públicos** de tres proyectos:

Proyecto	Bugs públicos	Descripción	Runner
black	2	Formateador de código (casi todo en un único archivo grande, `black.py`).	unittest
fastapi	2	Framework web (paquete con varios módulos).	pytest
scrapy	8	Framework de *scraping* (paquete grande).	pytest

El índice completo está en `dataset/bugs.csv` (id, proyecto, versión de Python, runner y tests que fallan).

La evaluación final se realizará sobre este conjunto **y sobre un conjunto oculto de bugs distintos de los mismos tres proyectos**, con el mismo formato, que no se publicará. Ambos conjuntos tienen una dificultad comparable. Todos los bugs de la evaluación, tanto públicos como ocultos, fueron verificados por el equipo docente: **son resolubles con el modelo y el límite de tiempo (3 minutos por bug) indicados en este enunciado**.

2.5. Herramientas disponibles y Stack Tecnológico

- **Lenguaje del agente: Python 3.14** (probado con 3.14.4), en un entorno propio (`agente/.venv`, creado con `python3.14 -m venv .venv`), distinto del entorno de cada bug. La evaluación se ejecutará con esta versión. Con pyenv deben instalar ambas versiones: `pyenv install 3.8.3` para los bugs y `pyenv install 3.14.4` para el agente (ver README).
- **Entorno de los bugs:** Python 3.8.3, creado por `setup.sh` en `repo/env`.
- **Agente LLM:** librería `google-genai` con `model: "gemini-3.5-flash-lite"` (plan gratuito de Google AI Studio). Durante el desarrollo, cada grupo usa su propia API key. El agente debe leer la API key desde la **variable de entorno `GEMINI_API_KEY`** y nunca incluirla en el código, ya que la evaluación se ejecutará con la API key del equipo docente. El archivo `config.env` del grupo no se usa en la evaluación ni se entrega.
- **Sistema operativo:** Linux, macOS o WSL. En WSL, trabajen dentro del sistema de archivos de Linux (`~/`), no en `/mnt/c`.

Pueden usar libremente herramientas deterministas dentro de su agente (por ejemplo, ejecutar tests en subprocesos, medir cobertura, analizar el AST, buscar texto en el repositorio).

2.6. Consideraciones prácticas

- **Cuota de la API:** el plan gratuito de Gemini permite aproximadamente **500 peticiones por día, por proyecto de Google y por modelo**. Cada ejecución completa sobre los 12 bugs públicos consume parte de esa cuota, en proporción al número de llamadas que haga su agente por bug. Prueben primero con pocos bugs y **no dejen las ejecuciones finales para el último día**.
- **Errores temporales:** la API puede responder 503 (saturación) o 429 (límite de uso). Su agente debe reintentar con espera creciente, sin exceder el límite de 3 minutos por bug.
- **Tiempos de instalación:** instalar un bug (`setup.sh`) toma entre 30 segundos y 4 minutos según la máquina; instalar los 12 bugs públicos, entre 10 y 50 minutos (se hace una sola vez).

- **Tiempos de ejecución:** los tests de un bug tardan segundos; los módulos de test completos, hasta algunos minutos. Con el límite de 3 minutos por bug, ejecutar el agente sobre todo el conjunto público toma como máximo 36 minutos.
- **Verificación local:** `herramientas/verificar.py` aproxima la evaluación oficial. Como ustedes no tienen el arreglo oficial, compara las regresiones contra los tests que pasaban con el *código original*; en casos muy puntuales su resultado puede diferir del evaluador del curso.
- **Generalización:** se recomienda separar algunos bugs públicos y no mirarlos mientras desarrollan, para usarlos como validación y estimar cómo le irá a su agente con bugs no vistos.

3. Parte II – Video Explicativo

En esta tarea **está permitido usar herramientas de IA para programar**. Por lo mismo, el video tiene como propósito verificar que el grupo **entiende y puede defender su código**. Deben grabar un video de aproximadamente **5 minutos**, con el código en pantalla, en el que ambos integrantes participen y **señalen dónde ocurre cada cosa en el código**. El video debe cubrir los siguientes temas; la distribución del tiempo y el orden quedan a criterio del grupo:

1. **Flujo general:** un bug concreto de principio a fin. Qué entra y qué sale de cada etapa, cuántas llamadas al LLM se realizan y por qué.
2. **Localizador:** qué señales usan, cómo eligen las regiones y cómo controlan el tamaño del prompt (piezas 2 a 4).
3. **Reparador:** cómo piden y aplican el cambio, cómo verifican que funciona (incluyendo regresiones) y qué hacen cuando falla (piezas 5 a 7).
4. **Diseño y calidad del código:** cómo está organizado el código, qué piezas de la Sección 2.2 implementaron y cómo, y qué alternativas descartaron.
5. **Resultados y un fallo:** tasa de éxito y costo en el conjunto público; un bug no resuelto, clasificado según la etapa en que falla, y qué harían para mejorarlo.

El video debería permitir responder preguntas como las siguientes:

- Muestren dónde deciden qué código enviarle al LLM. ¿Qué pasa si esa función tiene 2 000 líneas?
- ¿Qué hace su agente si el cambio propuesto no se puede aplicar o rompe la sintaxis?
- ¿Cómo sabe su agente que no rompió otros tests?
- ¿Qué pasa si Gemini responde 429 a mitad de un bug?
- ¿Cómo garantizan terminar cada bug en menos de 3 minutos?
- Si el proyecto usa pytest en vez de unittest, ¿qué parte de su código cambia?
- ¿Cuántos tokens gasta su agente por bug y dónde se va la mayor parte?

4. Código Base Proporcionado

Se entrega `Tarea2.zip` con la siguiente estructura:

```
Tarea2/
|-- README.md              instrucciones de uso, reglas, formatos y evaluación
|-- agente/                esqueleto que deben completar
|   |-- main.py            punto de entrada: localizar -> reparar -> escribir los JSON
|   |-- localizador.py     localizar(carpeta, info) -> {"regiones": [...]}
|   |-- reparador.py       reparar(carpeta, info, localizacion) -> {"resuelto": bool}
|   |-- requirements.txt   google-genai
|   `-- config.env.example plantilla para la API key de Gemini
|-- herramientas/          utilidades de apoyo (no se evalúan)
|   |-- preparar_todos.sh  instala el entorno de todos los bugs y comprueba que fallan
|   |-- verificar.py       revisa el resultado de un bug: tests, regresión y formato
    de los JSON
|   `-- ejecutar_lote.sh   ejecuta el agente sobre varios bugs y genera un resumen (
    CSV)
`-- dataset/
    |-- bugs.csv           índice de los 12 bugs públicos
    `-- <proyecto>-<n>/    un paquete por bug
```

Carpeta `resultados/`. Cada vez que ejecutan `herramientas/ejecutar_lote.sh`, el script crea automáticamente (en la raíz de la tarea) una carpeta nueva `resultados/lote-<fecha>/` con el resumen de esa ejecución y una copia de lo producido por el agente en cada bug:

```
resultados/
`-- lote-<fecha>/
    |-- resumen.csv            una fila por bug: bug;resuelto;llamadas;tokens_entrada;
    tokens_salida;segundos
    |-- black-12/
    |   |-- localizacion.json
    |   `-- metricas.json
    `-- ...                    una subcarpeta por bug
```

Antes de ejecutar el agente sobre cada bug, el script **descarta los cambios** que haya en su `repo/`, para que todos partan del código original, y aplica el mismo límite de 3 minutos por bug de la evaluación. La columna `resuelto` solo indica si pasan los tests de `run_test.sh`; no comprueba regresiones (para eso está `herramientas/verificar.py`).

Para la entrega deben incluir **solo la carpeta `lote-<fecha>/` de la ejecución de la versión final de su agente sobre los 12 bugs públicos**. Esa es la carpeta que usarán en el video para presentar sus resultados, y el equipo docente la comparará con su propia ejecución del agente.

Las funciones `localizar` y `reparar` solo incluyen su firma y lanzan `NotImplementedError`. **No se entrega infraestructura del agente** (ni cliente de Gemini, ni funciones para leer o editar código, ni prompts): todo el diseño del agente es parte de la tarea.

Para preparar el entorno de todos los bugs (Linux, macOS o WSL):

```
bash herramientas/preparar_todos.sh
```

5. Restricciones

- Solo se puede modificar **código fuente**, nunca archivos de test. Durante la evaluación se restauran los tests originales, por lo que modificarlos no tiene ningún efecto.

- El agente **no puede usar información externa al paquete del bug**: no se permite buscar el arreglo en internet ni en ninguna otra fuente fuera de lo entregado.
- El agente debe dejar la reparación aplicada en `repo/`.
- **Modelo obligatorio:** `gemini-3.5-flash-lite` para todas las llamadas al LLM. No se admiten otros modelos, proveedores ni cuentas pagadas.
- Localizador y reparador deben ser **etapas separadas**: el reparador recibe la localización como entrada.
- **Límite de 3 minutos por bug** para el agente, sin contar la instalación del entorno.
- No se permite código específico de un bug o proyecto (nombres de archivos, funciones o tests concretos, reglas pensadas para un solo caso).
- El agente debe ejecutarse de forma completamente automática, sin intervención manual.
- El agente debe reportar su costo real en `metricas.json`. Métricas falsas o mal contabilizadas serán penalizadas.
- No está permitido acelerar la grabación del video (por ejemplo, x2) para cumplir con el tiempo solicitado.

6. Plazos y Política de Atrasos

- **Fecha de inicio:** Martes 29 de septiembre de 2026.
- **Plazo de entrega (sin atraso):** Jueves 22 de octubre de 2026 a las 23:59 hrs. (duración: 3 semanas).
- **Uso de Cupones:** cada estudiante cuenta con un número limitado de "cupones de atraso" para el semestre. El uso de 1 cupón permite extender el plazo original en **48 horas** exactas sin penalización en la nota. Los cupones se descuentan por integrante: si el grupo decide usar un cupón, se descuenta 1 cupón a **cada** integrante de la pareja. Los cupones se descuentan automáticamente del sistema de entregas.
- **Descuento por Atrasos:** si se entrega tarde y ya no se dispone de cupones (o se agotan en la entrega), se aplicará un descuento de 1.0 punto (escala 1.0 a 7.0) por cada 24 horas o fracción de atraso.
- **Límite Máximo:** no se recibirán tareas (y serán calificadas con nota 1.0) después de 2 días de atraso respecto a la fecha de entrega original, independientemente de los cupones disponibles.

7. Modalidad de Trabajo

La tarea se realiza en **parejas**. Cada grupo debe usar su propia API key del plan gratuito de Google AI Studio (la cuota es por proyecto de Google).

8. Entregables

Basta con que un integrante del grupo suba los archivos a la plataforma. Si existen múltiples entregas, se revisará y calificará estrictamente el último envío realizado.

- El código fuente completo del agente (`.zip` de la carpeta `agente/`, sin `config.env` ni los entornos de los bugs).
- La carpeta `resultados/lote-<fecha>/` (`.zip`) de la ejecución de la versión final de su agente sobre los 12 bugs públicos, generada por `herramientas/ejecutar_lote.sh` (Sección 4). Los resultados deben ser coherentes con el código entregado: el equipo docente volverá a ejecutar el agente.
- Un enlace funcional y con los permisos adecuados para visualizar el video explicativo de 5 minutos (Parte II).

9. Evaluación

La evaluación se divide en un **40 %** para la implementación del agente y un **60 %** para el video.

Para cada bug, el equipo docente instala el entorno (sin límite de tiempo), ejecuta el agente del grupo con un **límite de 3 minutos por bug** y luego su propio evaluador:

1. Se restauran los archivos de test originales.
2. **Tests objetivo:** se ejecutan los comandos de test del equipo docente (no el `run_test.sh` del grupo); deben pasar.
3. **Regresión:** todos los tests de los módulos de test que pasan con el *arreglo oficial* deben seguir pasando.
4. Un bug se considera **aprobado** si los tests objetivo pasan **y** no hay regresiones.

Si el agente se cae o supera el límite de tiempo en un bug, este cuenta como no aprobado. Como el modelo no es completamente determinista, el agente de cada grupo se ejecutará **2 veces** y un bug se considerará resuelto si se resuelve en alguna de ellas.

La **localización** se considera correcta si alguna región de `localizacion.json` contiene al menos una línea modificada por el arreglo oficial; también se considerará si el acierto ocurre en la primera región.

La ejecución sobre ambos conjuntos la realiza el equipo docente después del plazo de entrega, con su propia API key; la fecha y hora de esta ejecución no se informarán.

9.1. Parte I: Implementación del Agente (40 %)

- **Cumplimiento del contrato y robustez (8 %):** los resultados entregados son coherentes con la ejecución del equipo docente; `python main.py <bug>` funciona sin intervención y dentro del límite de tiempo; `localizacion.json` y `metricas.json` son válidos y veraces; se usa el modelo correcto; no se modifican tests; el agente maneja errores de la API y del bug sin caerse.
- **Efectividad en el conjunto público (10 %):** tasa de bugs aprobados en el conjunto público.

- **Generalización (14 %):** tasa de bugs aprobados en el conjunto oculto, comparada con la del conjunto público.
- **Calidad de la localización (8 %):** aciertos de `localizacion.json`, independiente de si la reparación funciona.

El puntaje de efectividad y de generalización es **proporcional** a la tasa de bugs aprobados en el conjunto correspondiente. Por ejemplo, si el agente aprueba 9 de los 12 bugs públicos (75 %), obtiene el 75 % del puntaje de efectividad (7,5 % de la nota). El puntaje de localización se calcula de la misma forma, según la tasa de aciertos.

Penalizaciones. Si se detecta que el agente modifica archivos de test, el grupo obtendrá **0 puntos en toda la Parte I (implementación del agente)**. Un agente que detecta nombres de tests o incluye lógica específica de un bug o proyecto obtendrá 0 puntos en efectividad y generalización. Las métricas inventadas o mal contabilizadas (por ejemplo, tokens en 0 con llamadas mayores que 0) serán penalizadas en el ítem de cumplimiento del contrato.

9.2. Parte II: Video (60 %)

- **Explicación del flujo (12 %):** explica correctamente el flujo del agente y el rol de cada etapa.
- **Dominio del código (15 %):** es capaz de señalar en el código dónde ocurre cada pieza (selección de contexto, aplicación del cambio, verificación, reintentos, control de tiempo, conteo de tokens).
- **Calidad del código y del diseño (12 %):** presencia y funcionamiento de las diez piezas de la Sección 2.2, separación clara de etapas, legibilidad, uso de señales de ejecución (no solo "enviar todo al LLM"), verificación antes de terminar y ausencia de código específico de un bug o proyecto.
- **Justificación de decisiones (9 %):** explica por qué eligieron sus señales, prompts y formato de edición frente a las alternativas.
- **Análisis de resultados y de un fallo (7 %):** presenta la tasa de éxito y el costo del conjunto público, coherentes con los resultados entregados; analiza un bug no resuelto, identifica la etapa en que falla y propone una mejora concreta.
- **Formato (5 %):** respeta el límite de tiempo, tiene buena calidad audiovisual y participan ambos integrantes.